ActInf GuestStream 052.1 ~ A ElSaid, T Desell, A Ororbia "Ant-Based Neur

GuestStream | A ElSaid, T Desell, A Ororbia | 1:56:55

Hello and welcome. This is ACT-INF Guest Room 52.1 on August 10th, 2023. We have Ahmed Elsaid, Alexandra Arorbia, and Travis DeSalle. Ahmed's going to give a presentation and following we'll have some reflections and discussions. So thank you all for joining and Ahmed to you for the presentation. Okay. Thanks so much for having me. Today I'm going to discuss the methods that we came up to solve new architecture search and new evolution problems. The methods that are end-colony optimization-based solutions. So as the title here is mentioning, end-colony optimization for new architecture search and new evolution. This work is a collaboration between me, my previous advisor, Dr. Travis DeSalle, and my co-advisor, Dr. O'Rourke-Axander Arorbia. I'm currently an assistant professor in University of North North Carolina, Wellington. And moving on to the next slide. So as an overview of the things that I'm going to discuss today, I'll try to take, give a bird eyes view for what is new evolution, why we need it, and from there I'll try to discuss other methods that is, and that is based on end-colony optimization, which is called end-based neural topology search, or ENDS for short. And after that, I will discuss how we advanced with this idea by introducing a continuous ENDS, or CANDS for short, which is, which could thread off this free search space and replace it with a continuous search space. And after that, I'm going to also say what we did with the three dimensions and the continuous ENDS to have a back-propagation-free CANDS. And later, I'll discuss three of the points that we are considering for future work.

So, in the machine learning learn, as the structure, the neural network structures get, got deeper and deeper, people were trying to optimize the structures to have better performance. And people in different realms, or different problem domains, used to borrow the best performing neural architectures or structures, and try to modify a little bit to work for their problem. And they tried to tweak some of the features of the structure

and compare these different tweaks. And then they say that we found the best performing structure. But to actually find an absolute optimum structure, concerning that solution, it could be an NP-hard problem, because to reach that solution, they had to try all the combinations of the different structural elements. And because we have massive structures in these deep neural networks, it could be an NP-hard problem, because we don't have computational power or enough time to actually train and test all these structures, all these structures constructed from these different combinations of structural elements. So, the alternative way to do this is to actually try to do, to apply a meta-rearistic method, to converge to a neural-to-op-com solution. That is much better than not optimizing the structure or relying on a kind of like a random search, right? A random search can get us better performing neural network, but it's not going to give us a near-to-optimal solution, or to converge to a near-to-optimal solution. So, a meta-rearistic method would give us an automated method and also can converge to a near-to-optimal solution to this structural problem.

So, the way that people approached NAS was by trying to mimic how optimization is done in nature, right? So, the first method, the first way they thought of it was trying to mimic how living organisms evolve in nature, using genetic-based algorithms like the Darwinian genetic evolution. And it started with NEAT, NEAT, it's a short for Neuroolution of Augmending Topologies, and it relied on genetic algorithms, where also that concept was applied for in most of the NAS on neural evolution methods, and exam is one of them. Travis came up with this method, and it became one of the state-of-the-art methods in NAS. So, in such methods, we tried to again mimic genetic evolution by introducing new structural elements or removing structural elements or altering the structure through the evolution process through the evolution iterations, through the evolution generations. So, we can apply mutations by splitting edges or adding edges or disabling edges. And we display edge, we disable edges or disable some structural elements so that we don't lose that component, so that we can later on use it, kind of like a dormant gene in a genome, right, so that it can appear in later generations not to get rid of totally. So, we can, in mutations, we can disable edges, we can enable them later in later generations if we found that we want to try this option. We can also add recurrent edges or remove recurrent, or enable or disable recurrent edges. We can split nodes, we can take some of the nodes that in a previous generation, and we can just split it to two nodes and then take the edges connected to that node and try to divide it between the nodes that were generated from the previous node in the previous generation.
Also, we can add nodes from, to the structure, to the structure. So, all of these are part of the mutation process in the genetic process. We can also disable a node if we don't, if we want to try to, to just get rid of one of the nodes and disabling a node or multiple nodes will also disable the edges that connected to that node. Besides mutations, we, the other side of a genetic process is to do crossovers where we have two of the best performance population made together to bring an offspring, and offspring will have collection of the characteristics of coming from their parents.
So, it will take some of the characteristics from that parent, some other characteristics from the other parent, hoping that this will give us a better performing neural network, or better performing generation.
So, the main problem, sorry.
So, the main problem with, with, um, uh, genetic-based algorithms is that they start with minimal structure, like we can see there, meaning that inputs and outputs, and, um, you starting from the, the, the optimization with this minimal search space can track the method in a local minima through the optimization process. So, we were thinking how to get rid of this obstacle, and by having a, a bigger or larger search space to start with, and then we can sample some solutions from that, um, uh, larger space. And we were looking around and we, we, we, we, we, we considered end-colony optimization,
and, and I will, I will say why, that I'll try to introduce, um, the, the method first. So, the method was first introduced as, um, graphic optimization method, um, graphic optimization method, sorry, um, it was introduced in
mid-90s by Marco Durego. Marco Durego applied this method, um, uh, in, on a travel salesman's problem,

and the problem is mainly about a travel salesman who wants to visit a number of cities in the country,

in the country, you think it's Georgia's staff. And concerning this problem, if we have this number, uh, if the number of cities grows,

then the, the combinations of these numbers, uh, of these cities that we have to consider to find the open solution,

um, if this number of cities grows, then we will end up having an NPR problem, because we didn't, we won't have enough time or,

or, or, or computational power to have this, um, this compact, this exclusive, uh, solution, um, done,

or this exhaustive search done. So, he, he, from his observations, um, um, to, to answer in nature,

um, he found that he can apply how they forage to find food in nature, and, um, and, and, and then, uh, take this, this, this, this concept,

that this observation, and apply it in an algorithm to find the, not the optimum path, that leads from, um, from one point,

and to visit all the cities and in the shortest, shortest path. Um, so the method,

the, this observation, um, so the, the, the, the slide and the coming slides will just try to go,

and give us a picture of how, and, um, um, um, um, forage in nature to find food, and then how Marco Dorega took, took that concept to apply it, and, uh, in that Charles Hisman's problem. So, um, as observers found that, and go out from, from their nest to find food,

and, um, at the, the, in the different directions, right, um, and when they find food, eventually find food, they will take some of that food, and then they will go back to their nest, and in the way back, they will deposit some other substance called a furmon, um, that they, so they deposit that substance, so that they communicate that path to the food resource with other ants. So, actually, other ants do exploit this furmon, uh, other substance, and they, when they sense it, they try, they will,

they follow the path that the, the, the first ant took from the, uh, from the food resource to the nest,

and hoping that they will find food at the end of that path, and, and when they actually find food at the end of the, the path, they will take some of the food and do the same thing. They will deposit some more furmon on the same path, making it more appealing to other ants to take it, so that they can bring more food to the nest. So, the, this, in this process shows us the exploitation, uh, behavior of ants, but again, from some time, from time to time, other ants also try to exploit some other, uh, food resources, potential food resources, and they, they, they kind of resist, uh, following, uh, the, the, the formal, uh, traces, uh, and, and they try to find, to go away and find some, um, put it, new food resources for the nest. So, they, they are, they are not only, uh, exploiters,

they also explore them, and these two concepts were, um, used by Marco Durega to kind of, like,

ail ail

is not appealing anymore or the food resources are exposed, no more ants will take that path or when they take it and reach the food resources that is exposed, they will not take the same path to the nest again.
They will try to wander and to find more new food resources and because they're not going that path again and not depositing any furamone on that path, the furamone will eventually evaporate and disappears and making it less and less appealing for other ants to take.
So that's what Marco Dorego was looking at when he thought of the travel salesman's problem.
He wanted, he applied that for the travel salesman's problem by making one agent try these different paths and then comparing each through each iteration.
So that agent will take a path between the cities, right?
And then it will compare the length of that path to the previous experience with other paths.
And if it's shorter, it will try to deposit furamones on the segments of that path.
And eventually he was hoping and he was right about what he was hoping.
He was hoping that eventually the shorter path and shorter segments that give the ultimate shorter path had more and more furamone deposits, making it more and more appealing for the agent to take it through the iterations.
So we thought about this concept and we thought that it's very appealing to apply for an NAS problem because the solution was applied for a graph optimization problem and neural networks are in their sense directed graphs.
So we also considered an end client optimization because it's full torrent, decentralized and scalable, and it's also easily traceable.
And going back to being decentralized, it gave us, it made this method a perfect candidate for a parallel and high performance computing solution.
So, which will eventually accelerate the optimization problem.
And I think in the next slide, the slide after the next one, I'll discuss how we exploited or use this characteristic of end-colon optimization to accelerate the solution that we came up with, or the method that we came up with, which is ENDS and CAMES.
So, this scheme or the scheme of ENDS, applying ENDS or ENDS-colon optimization in a neural virtual, or sorry, in neural architecture search is depicted in or illustrated in this flowchart.
So, we start off with a massive search space expressed in superstructure, which expresses or embodies a neural network that is massively connected, meaning that each node in that superstructure is connected with other nodes through edges and recurrent edges, backward and forward and backward recurrent edges.
So, and then we let a number of agents swarm over the structure from an input node to an output node.
So, each one of these agents will pick an input node and then it wanders from that node through the connection,
recurrent edges and recurrent edges, and between the nodes and between the hidden layers,

till it gets and picks one of the output nodes.

And then we take all these paths of the different agents and we put them together to form a structure,

a neural network structure, and we take that structure and train it and test it.

And then compare its performance to the population of best performing neural networks,

best performing structures.

And if it's better than the worst in the population, then we reward the path that the agents took over

in the superstructure, we work with hormone, and so that it makes these paths appealing for later iterations

through the evolution process or the optimization process.

That's if the generated structure is best than the worst in the population.

If it's not, if it's actually worse than the worst in the population, then we discard that structure or that neural network,

and we don't reward any of the path that the agents took.

And also, the thermal evaboration will help us get rid of the thermals that were deposited on the edges

that are not giving us better and better structures.

And again, because the end colony is decentralized, we exploited this by having an asynchronous solution or asynchronous evolution.

We had a main process that took care of generating the new structures and also updating the population,

the best performing structures and updating the hormone on the superstructure.

So the main process would generate structures and send them to worker processes.

The worker processes will train and test the neural network on the data that we have for the problem.

And then send the results back or the fitness of the neural network to the main process.

And based on that fitness, the main process will either discard it or we take that this fitness and compare it to the best performing in the population.

If it's better than the worst, it will reward the path that ends took on the superstructure by the processing motor hormone.

Or if it's worse than the worst in the population, we'll just discard it.

And it will keep generating new structures and sending them to processes.

Because the training which relies on back propagation is the most computationally expensive part in this process.

If we have a number of worker processes that can work in parallel to train and evaluate these neural networks, these neural structures,

we can speed up the process by training and evaluating different structures at the same time in parallel in asynchronous evolution scheme.

This is actually an animation that's not working in this version of the slides because we're using a PDF,

but it's mainly a structure where you'll see edges or connections between these nodes,

fading or having darker colors based on the firm on values through the iterations.

So each frame in this animation is kind of an update for the firm on value of the edges based on the performance of the edges.

Based on the performance of the version of the neural network that were generated by the agents.

When they swarm from the start node, taking one of the input nodes in the little layer,
and then from there going to one of the hidden layers in this one hidden layer that we have here,
and from there going to the output.
So that was the concept of applying ATO or end-client optimization in NAS.
Now I'm going to talk about the actual method that we came up with.
So it's more generic and more powerful neural textual search methods,
more comprehensive if I may say that.
We opted to apply the method on recurrent neural networks because they tend to be potentially larger than other neural networks structures because of their current connections.
So we thought that if we started this problem, the method or the constant applies to a neural network,
but applying it to recurrent neural networks made it more appealing challenge to, for, for, for, um,
measuring the performance of the method that we thought of.
So this slide in the comment slide will discuss the different heuristics of the methods of ANS.
The third heuristic is superstructure itself.
And as we, as I mentioned before, it's, it's a massive search space and as massive as possible to be handled with the,
the, the, the machine or the hardware that we're working on.
Um, the superstructure consists of, um, a neural network that is massively connected,
meaning that every node in that structure is connected to the other nodes
via, or through edges, uh, forward edges and, uh, recurrent edges, backward, forward recurrent edges and backward recurrent edges.
Um, this simple structure that we have here represents one of just the concept of the superstructure that we have,
we apply in, in ANS. Here we have three input nodes, uh, three hidden layers, each have three nodes and one output node in the output layer.
And we are just showing one node connected to the other nodes through edges,
which are the ones representing green, uh, green, forward recurrent, sorry, um, the, the edges are, uh, the one in gray,
and then forward recurrent and backward recurrent in the green, uh, in the red.
And the concept of recurrent edges, um, might be a little bit confusing if we look at it in this example,
because how can an edge, um, be recurrent coming in and out from the same node or, or the, the, the nodes in the same time step.
But this structure might make it more, uh, clear.
Um, so here we have, um, a structure that's pretty much similar to the ones we saw before,
but we, here we have three input nodes, two hidden layers, there's three hidden layers,
each have three nodes, and then the output layer have two nodes.
Um, and then we have also three time steps, the current time step T here,
and the previous time step T minus one, the, the one before that T minus two.
Um, and then the, the, the, the, the solid, uh, black, uh, lines, um, and, and, and, of course, these edges are

present in the current time step that is going to propagate through the neural network.

And then the current, um, connections, these ones are going to bring information, um, or propagate information from the previous time steps, the previous, um, inputs or previous, uh, data that was, um, fired to the nodes in the previous time steps.

And these are current edges are depicted here in red and orange.

The four ones are depicted in, in red and orange.

The reds are coming from T minus one and the orange are coming from T minus two.

And the backward current edges are the dotted lines, uh, in, uh, blue and green.

And, um, and we can see that they are going backwards.

So they cut the, go backward and to backward layers.

Um, uh, but the, uh, but we, but because they are recurrent, we can do that because they are, they process information.

They bring information that's already processed.

So we don't have to worry about propagating information back through time or back through destruction, but we can do that if it's coming from previous time step.

Um, the second realistic in, in, in answer is then colony weight sharing.

We wanted to use, um, then, so instead of initial, uh, branding, initialize the weights or the snap that weights and generated, uh, neural networks.

Uh, we wanted to use the, uh, the train weights to initialize the weights of the, um, newly generated, uh, neural, uh, neural networks.

We did that by saving these neural networks on the superstructure.

Uh, we used the last, um, equation here to do this, uh, update.

So we balance between the weights that were saved previously and the weights that we were, uh, are coming from the train or, uh,

evaluated neural networks.

And we also used two strategies to do this update.

We used, um, a fixed parameter, uh, phi, or we let phi, that I, this, that was the first strategy.

The second strategy was to get phi by applying these two equations, uh, which relies on the performance of the neural network that was previously generated.

Sorry, sorry, the previously generated and trained.

So it relies on the performance of the neural network that was trained and, um, validated or tested.

So if, um, if it, if the performance is, was good, then we will let this, uh, the weights of that, um, uh, neural network to contribute to the, um, the, uh, to contribute more to the, uh, initialization of the weights of the newly generated RNNs.

If it's not performing that well, then phi will, this equation will not allow it to contribute that much.

The third meta heuristic is multiple memory cells.

So, um, each, at each node, um, uh, when, when, when, when, when an agent or an ant reach to a node in, in, in the superstructure, it will do, um, a local search, um, to pick, uh, the type of the neural network.

And then you're in the neuron or the type of the node, uh, from these three different types of memory cells.

So in the generated RNN, um, the generated structure, the nodes in the structure is not all the same.

They will be different based on the local search that, um, that the agent will do or the ant will do at each node.

They reach through their path from the input to the output in the superstructure.

The fourth meta heuristic is the multiple ant species.

So we applied, uh, different species, uh, or came up with different species for the ants.

Um, the first species was the ones that will traverse over the edges, only the edges.

So they would go only forward through the edge, uh, the, the, the edges of, uh, the neural network or the massive, the superstructure.

And these ones are going to define the number of nodes in the generated structure.

And also define the types of the nodes, uh, in the superstructure.

And when they're done with their work, then the social, or the second species, the social ants will traverse between these nodes.

But they will use, um, uh, the recurrent edges to do the, to do, uh, to move between these nodes.

So they will create then, uh, the recurrent edges for the newly generated RNA.

And we have two different species for these social ants, or these two different subtypes of species.

One is, um, the forward, so for the, so the forward, uh, social ants.

We, these ones traverse only over the, uh, the forward current connections.

They go from the input to the output, but over, only over the, the forward recurrent edges.

And then the backward recurrent edges, or the backward social ants, these ones go, go from the output to the input.

And they traverse over the recurrent connections.

And the way, the reason we thought about these different species is that we wanted to control the tendency of the ants to,

wander in, uh, around, uh, in the superstructure exploiting the, the, the, the convoluted, um, mesh of, um, recurrent connections.

Um, so we wanted to control that.

So we came up with this strategy so that we can just define the structure using the explorer ants.

And then the recurrent connections can be defined after that using the, the social ants.

So, um, the, the fifth, um, uh, meta-heiristic is the regularization of, uh, formal pavement.

Again, we wanted to, um, give the ants an incentive to bring, to bring, um, sparser, um, and, um, also well-performing neural networks.

Um, we, by just penalizing them if they bring, they, if they constructed, um, denser or bigger structures.

So we added this regularization term and, um, formula that updated the hormone value.

Uh, and as you see, the, um, the regularization term relies on the performance.

The data here is the performance of the neural network.

And it also relies on the size of the structure.

Um, the last, the second, last one is, um, is, is, is, is jumping ants, jumping ants, which is, we want to experiment with.

If we let the ants, um, um, if we let the ants, um, jump over layers when they move through them, through

them, the, the, the superstructure.

Um, if we let them jump over these layers to construct the neural networks compared to if we restrict their movement, uh, to jump one layer at a time.

How would this, uh, end up, um, performance wise, how, if they will give us parser and, the, the, well-performing, uh, structures, or they will, or this jumping will hurt, um, the performance, um, by giving us weaker, uh, uh, structures.

We could, you know, networks.

So, um, we, uh, use the time series data, uh, that can belong to coal-fired power plant.

Uh, we divided the data to have, uh, 7,200, um, records for training and testing.

And, uh, here the plot shows that the data and we can see that it's nonlinear, uh, um, and it's, um, acyclic and, um, and non-seasonal.

So it's hard.

It's a hard problem for a non, um, Newton, uh, neural network solution or regression, linear regression solution.

So.

The input consists of 12 parameters.

We, we, when we, we were trying to predict only one parameter, the flame intensity.

Experiments covered all of the heuristics of, uh, of hands, um, giving different values for these different parameters.

Um, the superstructure is, uh, consists of 12 input nodes, three hidden layers, which have 12 nodes, and one, one output node in the output layer.

Um, the recurrent connections span over, um, three time steps, one, two, and three time steps.

Um, in total, the superstructure had 49 nodes, uh, 924 edges, and almost 3.5 thousand, um, recurrent edges.

So if you unroll the structure over 30, uh, 72 time steps, uh, in, in, in back propagation through time, we'll have about, um, 3.5, sorry, 352, uh,

32, um, thousand nodes, about 6.5 million edges, and 20, about 26 million, uh, recurrent edges.

We, in the experiments, we also compared performance of ends, uh, using the same data set.

Uh, we compared it to exam and NEAT.

So exam is the, the state of the art in, um, neuro, neuro-artificial search, which is, um, genetic-based, um, methods.

We also compared it to NEAT, because it's, uh, like a benchmark in the neuro-evolution and NAS, uh, realm.

We, and also we compared it to, uh, fixed structures, unoptimized structures that, uh, were one, two, and three, uh, that had one, two, and three hidden layers, and also, uh, different types of, um, uh, memory-based cells.

Um, the experiments covered 16 on these experiments to cover all the combinations of the meta-heuristics, of the heuristics of the, of ANS.

Um, each one of these, uh, experiments was repeated 10 times for, uh, uh, for statistical analysis.

Um, ANS-gerated, well, 2000 RNNs for each experiment, each train for, uh, 10 APICs.

In, in total, ANS-gerated about, train, generated, trained and evaluated, um, 32 million RNNs.

It took a month and, well, uh, a thousand CPUs to finish the experiments.

Um, the results that we got showed that ANS outperformed, of course, outperformed the unstructured, um, unstructured, uh, structures, uh, unstructured, uh, sorry, unoptimized, unoptimized structures.

And it also outperformed, uh, NEET.

And then some of the combinations of ANS outperformed exams.

So, exam here is the fourth from the left.

And we can see that, that the, the mean absolute error for some of the versions or the combinations in, of ANS heuristics, uh, outperformed exam.

So, we, we tried to look, to do some physical, um, study for the results we got from ANS.

So, we tried to look at, um, the top, uh, performing neural networks coming from, in our results.

So, we tried to look at the, the top 10, 25, 100, and, uh, 250 and 500 results.

And we, to look at, um, the contribution of these heuristics and these best performing, uh, neural networks or structures.

We found that, um, these heuristics contributed effectively, uh, in most of these results.

Um, but the more, the, the thing that was really, um, um, intriguing for us or, or, or, or surprise us is that, um, the, that we saw that, that the, the, the recurrent connections, um, disappeared in these results.

Because the best performing neural networks, um, didn't have that much of recurrent connections.

Um, we, that meant, meant for us that, uh, the, the, the memory-based cells did the job for recurrent information coming from previous time steps.

But we wanted to, to expand on this later, but, so, it's on our list on discussing, uh, on, on, on our future investigations.

Uh, so, this is just a summary for, um, the achievements of ANS, based on the results that we got from our experiments.

So, ANS was the first metaheuristic method to involve, uh, the core of ASTO, or anticholent optimization, and recurrent neural networks, um, NAS, or neuro, and neuroevolution.

ANS, as heuristics control ANS tendency to wander around superstructure, exploit, exploit, and recurrent connection proved successful.

Because, um, the, the regularization method component, or the regularization heuristics from, uh, uh, uh,

it mean, give us better results, uh, showing here in this, um, table.

Um, and also the jumping ANS, jumping ANS here, gave us better performance compared to non-jumping ANS.

Okay.

And, uh, and the regularization also, um, uh, also the, the, the, the, uh, data, the, um, um,

weight sharing, uh, strategy also proved, um, effective when we, if we look at it, the results here compared to if we don't apply, uh, weight sharing.

So, all-tropormizing and, and strategies, uh, are generic, are generic.

So, the, the, the, the, the, the, the strategies that we use are generic enough to apply it for any, um, any problem or solution that is ant colony based, ant colony optimization based.

Um, the Fremont deposition, uh, that, the method that we came up with is also an evaluation.

And also, um, it wasn't introduced in any literature, previous literature, um, and the performance of ants compared to the other, uh, benchmark and state of the art in the realm is also, um, remarkable.

So, going forward, we thought that ant was, did give us a good, good results, but, uh, the main drawback of ants was that the, the, the, the discrete search space.

Um, so ants worked on that this massively connected, uh, uh, method massively connected superstructure, but it's massive, yes, but it's still, uh, discrete.

Um, ants can move freely between these, um, nodes and, or, or, or they are forced to move over between these nodes over these, uh, predefined, uh, connections, whether they are, uh, for edges or recurrent edges.

So, um, we thought of, of removing that continuous search, uh, discrete search space and replacing it with a continuous search space.

So, we designed a 3D, uh, search space, uh, where, um, the, the, the, the search space had, like, layers representing the lag, the time lags.

Um, and then ants can jump over between these, uh, these, uh, layers to, uh, have, to give us the, the recurrent connections.

And on each of these, um, layers, ants will just give us the forward, the forward, uh, sorry, the edges between the nodes and the edges between the nodes.

Um, so in this slide, in the comic slides, we'll show an example of how ants move in camps or continuous, um, or continuous ants.

Um, so the, an, an agent or an ant will just start by picking up one of the layers that will move on.

This is done in a discrete fashion.

And once this, this is done, it will then decide if it's going to do an exploitation or exploration movement.

And in this, um, example, we, it decided to do an exploration movement.

So it will, um, decide the angle and the radius of its next allocation on, on that layer.

Once that's done, it's going to go forward and to that location, and then decides if it's going to do an exploitation or, if the next move will be exploitation or exploration move.

And this example is, it will be an exploitation move.

So it will try to, uh, exploit the pharomone traces, the pharomone that was previously deposited by other ants on, in the search space.

So it will use its sensing radius.

That's something that was, is previously defined for each ant.

And then it will, um, find the center of mass of the pharomone traces.

And then once it finds that it, when it calculates that center of mass of the pharomone, it will consider it as its next location.

And then it will move to that spot.

And then it will decide about if, if, if its next move will be an exploitation, exploration move.

And it, and, and at each location before it's deciding if about the type of the step, if it's exploitation or exploration, it will also decide if it's, if it's going to stay at the same level,

at the same time lag or jump on, uh, to a next, uh, time lag.

Um, if, if, if the jump or the movement is on the same level, um, um, if the, if the movement is, is on the same level, the ant, when, if the ant is doing an exploitation movement, it will only consider the pharomone traces that is, um, um, ahead of it.

Um, because it can't move backwards on the same time lag.

Otherwise it will be doing a backwards step, which is not allowed in, in, in your networks.

But if it's going to, um, um, another time lag, um, above, um, above layer, then it can be, it, then it's going to do a recurrent edge.

And, and, and, and so, recurrent edges can go back in time.

Oh, sorry, back in the structure in, in the previous type, uh, uh, layer.

So now the, the ant can consider all the pharomone traces within its sensing radius, the ones that are ahead of it and the ones that are behind it.

So in this example, he is going to consider, um, in this step is going to consider all the pharomone traces within its sensing radius, calculate the, the center of mass, and then consider it as its next, um, position.

And then keep doing, do, it keep doing this till it reaches to, um, the proximity of the output nodes.

And once this is done, it will consider, it will decide which time, which, uh, output node it will consider as its final, um, position in this path from an input to an output.

And, um, then, uh, other ants will do the same.

And, uh, then we'll have different paths from some inputs and some outputs.

And then accounts will take these paths and then try to convince, condense, um, the nodes so that we don't have so much nodes and, and that are very close to each other.

So the one nodes that are within certain proximity will be, uh, clustered together using DV scan.

Um, so have less, less number of nodes.

And then the path will be, the paths will be taken to, uh, collected and put in a structure, a neural network structure, and, um, then sent to a worker process to train and test and then compare it to the, the population, the best performing, uh, RNNs.

And the process will be almost the same.

And from this point, it will be almost the same as ants.

Once the structure is constructed and the training and testing process, uh, will be the same as the, the ones that we discussed in ants.

Um, so this is, this was also an animation, which shows, which shows, um, the, how these paths are taken by ants, uh, look, uh, from an input to an output in the 3D space.

It's 3D search space.

So ants, if we look at ants, uh, the ants have, uh, only eight, uh, tunable hyperpaneters compared to, when comparing this to ants and the XM, it's half of the number of, uh, hyperpaneters in the other methods.

These hyperpaneters are the number of layers of the search space, the number of time of ants, how much, how many of them, um, we have in search space.

Number of agents, number of agents, number of ants, which is similar to what we, similar, we have a similar, uh, hyperpaneter and ants.

Um, the, the sensing radius of the agents or the ants, um, the, and the agent's probability to create a new node, uh, which, uh, presents the exploration,

sort of the exploitation of the parameters or the form of, uh, that traces in the, in the space.

Um, node compensation, um, parameters or, uh, uh, factors that, the, the, the, the, the, the variables that, uh, representing the DB scan are considered also, uh, hyperpaner, hyperpaners in, um, in camps.

Um, from one updating pattern and from one volatility pattern, and these are also present, present in, um,

ants and, and, and ACO based, uh, problems, uh, solutions, sorry.

Um, the experiments used three time spheres, um, different number of panels and different sizes.

Um, the results showed that ants, um, um, the, the, the, the results that we got from cans were very, uh, competing with, um, ants and exam.

Um, they weren't necessarily, um, better, but they competed, they were at the same level.

Um, but it also, uh, did not, didn't do very well, uh, in one of the database, the, uh, datasets.

But it, uh, comparing the, the, that was comparing the performance as the mean absolute error, but, uh, comparing cans results, uh, from, um, the, um, sorry.

Uh, from the size of the new networks that we got from, from cans, we saw that, um, cans, uh, structures were, um, sparser.

Um, so the performance was competing with, um, ants and exam, but the structures were also, were compared, compared to ants were much, uh, sparser, uh, similar to exam, uh, the, the, the size of the, the structure that we're getting from exam.

Remember, exam is, um, genetic based, meaning that we start the structure, um, start the optimization process using the minimal structure elements.

Um, but it was susceptible to, it was susceptible to local minimus.

Trap, traps, um, ants, tents, is not susceptible to this problem, but it also gave us, um, smaller, uh, structures, uh, with performances that were competing with other methods at exam and ants.

So, the, the advantages of, uh, cancels is that it's, uh, it has an unbalanced surface compared to ants.

Um, results, results compared, uh, the results that came out, uh, were good compared to ants and exam.

Hyperparameters, the tunable hyperparameters are half of the, of these in exam and ants.

Um, and also, in that they encoded the neural topology to 3D search space.

So, this is one of, um, maybe, um, the, um, the contributions, um, important contributions of CAMHS.

Um, then, so, so far, ants and CAMHS are solutions, um, that applied neural, um, uh, to call, uh, architecture search, meaning that they did not optimize the synaptic parameters of the neural networks during the optimization process or during the evolution process.

Um, we, um, we thought, um, we thought we thought we thought we thought we thought we thought we thought we thought we thought we thought we thought we make ends capable of doing this as well to train the neural network or optimized sync

in that map these parameters in that new dimension.

We also wanted the ENDS to be self-aware and try to evolve themselves through the evolution process

so that they can adapt to the changes or adapt themselves so that they can give us better performance.

We wanted them to change their behavior of their characteristics like the sensing radius they had before, for

example,

that can be a variable that can change through the iteration thing and evolve through the iterations.

Each ENDS can change these characteristics as the evolution goes on progress based on the performance of the neural networks that they are generating.

So the advantage of doing this was that it eliminated the backpropagation process, which is the most computationally expensive part in the evolution process,

in the methods that we use so far in the XAM, ENDS and CANDS.

So eliminating the backpropagation gave us much, much faster evolution process.

The results we got, I will discuss the graph on the right-hand side first,

which discusses the fitness or the MS main absolute error of the results we got from backpropagation-free CANDS,

the four dimension CANDS compared to the normal CANDS, the backpropagation CANDS and ENDS.

So the results showed us that, again, on a particular database, that backpropagation CANDS and backpropagation-free CANDS did a quite similar job,

but they were both better than ENDS on this particular database.

But the main contribution or the main advantage of applying CANDS shows up in the graph on the left-hand side,

because we can see that if you compare the results based on the time of the evolution,

based on the evolution time, we see that CANDS was much, had, they took much, much less time than the backpropagation version of CANDS and the ENDS.

using these different number of CANDS.

Also, this graph shows how fast backpropagation-free CANDS compared to the normal CANDS.

So, in this figure, the curves in the dotted lines are, shows the time that backpropagation-free CANDS and CANDS took to prepare or generate the neural networks.

We can see that backpropagation-free CANDS took more time to prepare or generate neural networks compared to backpropagation CANDS or the normal CANDS,

and that's because it has the fourth dimension also has to evolve the ENDS or the agents through the iteration.

So, it takes more time to do that.

But, the other curve, the other two curves in dashed lines here are the lines that shows the time, the amount of time it took for the two methods to validate, train and validate the neural networks.

So, backpropagation-free CANDS doesn't have to train the neural networks, doesn't do backpropagation.

So, you can see that the time it took is in an order of magnitude less than the backpropagation version of CANDS.

And, these solid lines shows the cumulative time it took for both methods.

And, of course, it shows that backpropagation-free CANDS took much, much less time than backpropagation CANDS.

The future directions that we are, the future points that we're considering for, sorry, the point that we're considering for future work are to turn CANDS to a complete continuous search space.

So, as you saw, we, the, the, the, the, the search space for CANDS is not purely continuous because the

time lengths are represented by discrete layers.

We want to replace this with the continuous dimension, continuous layer representing the lag, the time lag in, in RMS.

However, this is a little bit challenging because the, the time lag has to be known prior to any optimization process because other than that, we, we, we, we will be mapping the whole, we, we, the whole time series as time lags.

And then picking the time lags from them, which is not feasible.

So, this is the first thing we, the first point we considering to investigate for in future work.

The second one is to investigate this finding that we, we, we, we had, we found in, in, in, in, in the results in ANS where the recurrent connections disappeared from the best performing structures.

And the theory that we have is that the memory based cells replace, replace these connections to give us the information that we need from the past nine steps.

And so that they were more effective, more efficient in doing this, compared to their connections.

So we have to do this, get expand on this and this gain more.

The last thing we want to consider also, I mean, this is one of the top things that we want to see.

These three points are the, what the top on our list.

The third one is to actually consider one of the, the, the concepts that was coined in a book by Dr. Deborah Gordon, which we, where she mentioned that the living organism in, in ANS world is not, are not the ANS themselves, but there are the colonies.

The colonies are, are the organisms that are, the, the, the, they start and they grow and, and they interact with the environment of the ecosystem.

They interact with them, with, with other colonies and they die at some point.

And the, the ANS themselves are not the organisms.

They are the cells of these organisms, these colony organisms.

So we want to take that concept and apply it in our methods to have number of colonies living together in parallel, evolving and communicating with each other.

So that we, and we want to see if this would give us a better performance.

But because after all, we're trying to mimic nature in our, in the, in the, in the solution we, we, we, we, we're trying to investigate.

So with this I'm done with my presentation.

And if you have any questions.

All right.

Awesome.

Wow.

What a great presentation.

How about Travis and Alexander, either of you, which want to go first, please feel free to give an introduction and any primary remarks that you have on that.

Okay.

I don't have too much to add.

I think Abdul Rahman did a pretty good job, you know, calling over the core bits of the work.

I'm Alex Rodia.

I'm an assistant professor in computer science, an affiliate faculty, affiliate professor in psychology and affiliate faculty in computational neuroscience at the Rochester Institute of Technology.

And I work on a lot of stuff, but primarily predictive coding, active inference, variational free energy, a lot of the stuff that's actually of interest to this group.

And yeah.

And this was a particularly interesting project for me because a branch of my own research is working in neuro evolutionary methods, or even just nature inspired meta heuristic optimization.

And, you know, when I got the pleasure of working with all the ramen when he was a PhD student at our IT, you know, we, you know, we talked a lot about the end colony based optimization approaches.

And I encouraged him to do to look into sort of like the origins and also try to understand actually how physical ants behave.

So that was always fascinating to me.

I don't have too much to add in terms of the technical parts.

I think he covered all the core results.

The only thing I, I like to think about it and I'm actually more fascinated to, to hear from the active inference Institute, their interest in the end colony methods and particularly what was interesting to them because thinking about what does end colony optimization, how do you view it from like an active inference perspective is I think particularly interesting and thinking.

And I even had a thought, I don't know if this was a question among the audience is mine.

Do the little ant agents or the can't agents that I'll go run and described earlier.

Um, you know, what, is there a way to start to view them as like a multi agent system that's optimizing some variational free energy quantity.

Uh, and you know, cause you know, a lot of even like Carl's work, I mean, I collaborate with him, you know, sort of touches into this area of like, well, what happens with collective intelligence and societal organization.

And you can kind of look, look at free energy from these very, very high level viewpoints all the way down to fine grained cellular activity.

And so, uh, I'm actually more curious to know.

And Daniel and anyone else in the active inferences who can sort of explain their particular interest in and based optimization and meta heuristic optimization.

I'm curious to know that, but there could be some interesting viewpoints of, you know, like what is the free energy.

I think the ants themselves are very simple.

I mean, we have made them more intelligent.

I know all the Roman and I talked at length about, well, what if we even made them, for example, have like a reinforcement learning control system.

And you could even imagine, well, now what if the ants themselves, uh, you know, uh, engage in a form of active inference themselves?

What would that look like for the system?

And they are optimizing their own free energy.

Uh, those are just fun little thought experiments.

I, we have obviously not worked on them.

Uh, at least of the Raman was never exposed by me to that part of my world.

Uh, and, you know, and, uh, biomimetic intelligence.

So those are my comments.

I'm not sure if they're particularly helpful, but they're very general.

Uh, and I'm actually more curious to know from the active inference Institute, uh, their interest in it.

And, you know, where does that maybe perhaps intersect?

Or this is just like, oh, we like, you know, interesting topics and intelligence.

And so, yeah.

Thank you, Alexander.

Travis, you want to say hello and give any reflections on the talk?

Sure.

You're all coming.

Hopefully you can see me a little bit.

My, my jungle here.

Um, hi, I'm Travis, I'm an associate professor at RIT.

I'm also, also the graduate program director for our masters in data science.

So if any of you are interested in any of this or data science, please, you know, shoot me an email.

Um, no, I thought this work was really very interesting in a lot of ways while Abdul Rahman was working on it.

Um, I think one of the coolest parts about it is a popular neural evolution algorithm.

Um, that came after neat is called hyper neat, and it transforms the, um, discrete search space of neural architecture search into a continuous one.

Um, and it's, it's shown to be pretty, a pretty powerful method.

Well, this, this version of ant colony optimization, the new one does the same thing that makes the search space continuous.

But what I found was really cool about it was that, um, as opposed to traditional ant colony optimization, where you have a graph and you just send the ants along the edges of the graph and you can, and you take the best paths and construct a graph after it.

Um, here the answer actually working like ants in the real world where they'll move a continuous amount of space, not just from point A to point B, um, and actually dropping down pheromones.

And it's more like a real simulation of how ants would move around in the real world.

Um, and we're getting better results from it than some of the old, older methods.

So, um, I think that is, that really made me be kind of happy to see that and thought it made it as very, very interesting work.

That's awesome.

Um, Ahmed, want to add anything, or I'm happy to give, give a thought on the ants and ask some questions

from the live chat.

Oh, I'm good.

Um, Travis, I think covered all of the things that we, I wanted to say, um, about, uh, um, I think just one thing that Alex mentioned.

That we give higher intelligence to the agent.

This is something that we discussed and I'm pretty much, uh, open to that.

I've just started actually to think about it, but I didn't implement anything yet.

However, the, the, the last thing that I discussed in the future directions is something that I actually started working on, but they didn't start the experimentations yet.

So hopefully we will see something out of that.

Awesome.

Well, there's a ton of ways to go.

And isn't that kind of one of the fundamental questions like in an interactive setting, either pure agent stigmergy interaction or multi-agent, but ultimately mediated through multiple stigmergies with like reading and writing and error correcting codes.

So in that communication setting.

Um, I felt like the work generalizes along multiple dimensions that previously approaches to multi-agent just didn't have those kinds of flexibilities.

Like the continuous time feature and, and, and several other features.

Um, I guess with respect to the ants themselves, um, I did five summers of field work with ants in the USA Southwest in Arizona and, uh, observed a lot of foraging activity.

So that problem or that context or setting is really a fun one and a pervasive one across any kind of living system, anything that's going to be active and, uh, living.

So why did you pursue foraging type algorithms overall?

And does this class include the interaction based methods with direct agent contacts that professor Gordon highlights in the ant encounters?

Yeah.

I, I, I, I, I, I will try to respond to this, but I, I just wanted to add to mention that the bridge

thought about this map actually, Travis started this idea about using and colonization and more

pollution or new art sector search by applying this method in simpler neural networks, uh, Ellen

and Gordon neural networks, you can learn that.

And I took the leap from there and started working on my pieces, my, my, my, my PhD pieces.

So, uh, we, we thought about this idea.

I think I, I mentioned earlier about why we used, uh, in contact optimization in previous slides,

but just repeating that for the audience to make sure that I, they got, we thought about

this idea because, um, and contact optimization was applied for, um, as a graph, uh, graph base,

uh, graph optimization solution.

And we thought that why not neural networks since they are in their sense neural networks.

They are directed graphs.

Neural networks are directed graphs because the flow of information goes from one direction to another direction.

So they are directed, but they also are graphed because they are nodes connected with the edges.

And our ultimate aim or goal is to optimize the structure by removing and adding elements to it so that it gives us better performance.

Awesome. And what one way that Professor Gordon and others have have talked about that bi-directional learning relationship between the computer science and the math and the analytical formulations and then the fieldwork and the actual behavior is because the thousands of species of ants, they're working amidst a huge range of ecologies with all these different patterns of regularities, all these different resource distributions.

And foraging, it's amazing how general it is, yet it's also just one of the functionalities that need to occur in terms of these like even slower processes, like allocation of tissue to faster processes, even of like response to alarm pheromone.

So it's like this one class of algorithms clearly scales across from view.

Ultimately, one, some of these foraging algorithms are lone foragers.

They don't leave pheromone trails.

So it's like even the idea of leaving a single positive pheromone or leaving them then.

But also there's things models can do that ants can't do, like the time travel pheromone, lossless perception, high dimensional signaling profiles that can't occur just with like finite amounts of molecules.

And now they are just persisting on their path as active agents within one generation with a variational free energy at the behavioral scale, then across generations with that evolutionary layer and the relationships between the neural network implementation and the active inference model.

ail ail

I think this might be more of an Alex question, to be honest.

Well, yeah.

I mean, if you want to keep it on active inference, but I think, Travis, you're going to need to tag in when you want to get into the very specifics of, like, the actual ant colony details.

Because, again, I kind of see the ant colony optimization from a more global point of view.

I would say, so to be clear, I mean, this particular work that of the, so that way of the realm, it's not also completely blindsided by your question, Daniel, even though the name is in the institute itself.

This isn't an active inference work.

So, you know, again, while there are, obviously, as you pointed out, lots of interesting elements, like, for example, the fact that the ants, when they conduct their exploration along, let's just think about, like, the recurrent networks and they're figuring out what nodes and what, as Abdelrahman explained, the superstructure, right?

As they iterate across with the pheromone trails and figure out what nodes I want to recurrently connect,

B-forward connect, skip connect, so on and so forth.

You would say, well, okay, what these ants are doing is they're engaging in epistemic foraging, which is a key concept in active inference, right?

So that way, Abdelrahman and Travis are not also left behind by jargon, epistemic foraging and active inference.

It's a big general framework.

It's like a biological, neurobiological process to RL.

And epistemic just refers to kind of like the uncertainty, Travis, that you and I work on.

And the idea is it's saying, well, okay, I want to understand my world.

And the more that I explore my world, right, there will be things that surprise me less.

But if I encounter some information that is really weird when I build a generative world model or a predictive world model, that's very surprising.

I should probably explore that.

And so, of course, I'm condensing the concept down into sort of like the exploration part of the explore-exploit tradeoff that I know you know, but that characterizes reinforcement learning.

So that's just what we mean when we say epistemic or epistemic foraging.

And obviously, Abdelrahman foraging, you know, can be likened to what the ants are doing, right?

They're exploring their environment.

And so, I guess with that in mind, so that way everyone's sort of on the same page here, to get to your question about the differences, about how does this work versus like your typical neural-based approaches to active inference.

And I would fall into that category of, oh, I build neural models, biological process models.

Those are very much focused, you could say, at the individual level, at least the ones that I am aware of.

When you're building, for example, even a back-propagation-based partially observable Markov decision process in, you know, active inference, that's like a single agent, right?

You're trying to build this construct that is trying to, you know, balance the epistemic quantity with its instrumental term, which, by the way, another jargon term for you, Abdelrahman and Travis.

Because that's just like your reward signal or your prior preference or a prior distribution over goal states.

And so, the agent, these agents sort of like deal with that trade-off, but at an individual agent level.

Now, again, I'm sure that there's interpretations of these from other perspectives.

The ant-based approach, even though I would not argue per se that this has at least an explicit form or a connection, at least that Abdelrahman has made clear to active inference.

But the idea is that this is like a multi-agent approach to active inference.

And so, the ants, when they conduct their epistemic foraging, which arguably is a very, very simple model.

Each ant and of themselves is, you know, essentially a bunch of, you know, coefficients and some hard-coded rules.

Because their job is essentially to work together with their pheromone trails to figure out, oh, what parts of the superstructure are useful.

So, I'd say that that's different.

And it lends itself in some ways.

Of course, you can make the ants more complicated and lose the benefit.

And I'm just about to say you could massively parallelize this.

And this is one of the key strengths that I think is natural.

And, for example, a lot of actually nature-inspired optimization algorithms and ant colony optimization is one of them.

You know, Abdelrahman has been using hundreds of CPUs.

And you can put these ants on their own individual processor.

And the communication that's, you know, occurring across them as they exchange information is, you know, through the pheromone trails.

There's an indirect mechanism.

It's not terribly complex to facilitate.

And I'm sure that there's even better ways to go about doing, like, asynchronous forms of communication.

And it further, further optimizes.

I know Travis, and Travis can add to this, has done things on, like, citizen science and distributed computing through, you know, the, you know, volunteer, volunteer computing.

And how you can distribute this through a massive global asynchronous network.

So you can imagine adapting the ant form, sorry, the ant colony optimization approach to some, like, distributed, massively distributed version of active inference.

Where you essentially have to write down that the variational free energy.

And I'm putting quotes around this because, again, there is no concrete term written in Abdelrahman's work.

I mean, because we haven't, at least we haven't viewed this from the active inference perspective directly.

Each ant is optimizing its own variational free energy.

But then there's probably a global quantity that sort of is relayed as a function of those pheromone trails to the individual ant agents.

And then, of course, with the exploitation term or the instrumental or the, you know, what is the reward signal to give an RL term.

That's sort of driven by the performance of the actual agent on the, each candidate agent on the task, right?

Abdelrahman, you compute, like, mean squared error when you're doing time series prediction.

So, in some sense, we have built in, you know, a reward function that we use.

And, again, for those in the active inference group here, you know, you can use the complete class theorem and look at, you know, the prior preference of saying, oh, well, the reward is actually technically a prior preference, right?

It's like a log probability.

So, with that in mind, you could squint at ant colony optimization.

And I guess the big benefit comes from that massive parallelization that you wouldn't actually very easily, if at all, get with, you know, our single agent neural-based approaches.

And that might be an interesting place to build on.

And I'll stop rambling at this point.

I'm not sure if that was helpful.

That was awesome.

Even earlier today, Chris Fields in the Physics as Information Processing course was talking about the classical information inscribed on the blanket, which is, like, the pheromone perception and deposition, pheromone modification and perception, sense-making and action, which we can associate with, like, the Nestmate cognitive system.

So, then there could be as simple as a pass-through for the Nestmate.

Could be any arbitrary relationship described by the blanket.

Simple Nestmate, sophisticated Nestmate.

Like, another level of time series modeling, whereas there's the environmental time series modeling.

And that's just in the ants.

And then the fourth dimension is, like, that quantum rotation, which goes from the lower dimensional classical stigmatic screen into the quantum informational space.

And so, that's one of the discussions ongoing in ACT-INF right now is about, well, previous approaches to connect quantum formalisms to macro, let's just say, neural phenomena based it upon the plausibility of, like, a molecular electronic bubbling up.

Whereas, just with research from decision-making and statistics and just multi-perspectival modeling and all the issues associated with the physicality of information transfer, the finiteness of it, the quantum formalism becomes useful just by itself with or without reliance on some other electronic phenomena.

So, it's just a lot of very interesting connections, like having the degrees of freedom on the blanket, which could be noiseless and forbids, or it could be noisy with this really specific thing.

But in silico, you get to play it from both sides and scale things up and down and do these meta heuristics on top of that arbitrary space.

It could be really simple for learning, or it could be however much.

And then, just like the ant colony algorithm is ultimately federated through embodiment, that property makes it a really useful candidate for biomimicry.

So, a lot of times when people think about collective behavior, they're thinking about, like, the flock of birds and the school of fish.

And those are, of course, collective systems and collective behavioral and all these kinds of, like, complex systems properties can be studied in that type of system.

But it is also neglecting at least an analytical degree of freedom with the stigmagy.

So, that really opens up that, both the quantum and the classical information, or both niche modification and behavioral and cognitive modeling.

So, just to add on, because I think, and then also, what the node is could even be heterogeneous or unknown, or fit in different ways, or fixed through design processes.

So, just like the ant colonies are flexible, enabling them to live in all kinds of places, make all these kinds of nested decisions that interact with each other.

That flexibility is just like the tip of the iceberg of what we could even just describe.

Because there would always be real environments we hadn't yet tried with ant colonies.

So, we really would never know the full extent of, like, all the repertoire and the dynamics of the ant system.

But then, we can just abduct into new mathematical, statistical, distributional frameworks, pull back to different levels of the learning and the meta-learning process, and just start there.

And then, almost ironically, or maybe the opposite of that, it could be applied to ant colony video data, or movement data, or foraging activity itself.

But, it kind of takes inspiration and develops in parallel or in conversation.

So, it's not, like, balance what real ants can do.

Or, you could constrain it so that there are properties that real ants have.

Like, they can only interact within this certain way, or, like, there really are only this many pheromones.

Do model comparisons.

So, it's a lot of degrees of freedom.

I feel like you all are opening up with the ant colony modeling.

And, also, one of the challenging pieces of multi-agent simulation is kind of, like, the open-endedness with the design space.

So, then, it's very hard for even creative ideas, like, sometimes to find the right compute resources that, obviously, are still even needed for what you discovered with the analysis here.

I'll ask a question from Bert in the chat.

I just want to, before we move on to that, I just want to clarify, just so, make sure that I got the right term.

And, certainly, Abdulrahman and Travis might want to look it up.

Is, when you were saying blanket, you were referring to a Markov blanket, correct?

Yes.

So, the technical definition of Markov blanket is, when you have a Bayesian graph, where nodes are the variables and edges are relationships amongst these variables,

for any given node of interest, we'll just call it internal states.

So, these are not features of the world, it's not tagged on to some tissue of a real nest mate in the world.

This is something that's tagged on to, or, like, a perspective we could take on any node in a Bayes graph.

And then, all the nodes that insulate it, and the co-parents, are known as the blanket, in the sense that it's, like, one-layer insulator.

And there's a lot of more discussion on it, and the philosophical implications, and all of these generalizations of that.

But, broadly, the Markov blanket is just the inbound dependencies, which we associate with sense-making and perception, learning, attention.

And then, the outgoing dependencies for the agent, which we associate with, like, action, influence, in some downstream pointing way.

Thank you.

I just wanted to clarify that, because I don't think Abdelrahman and Travis might be familiar.

Just with that terminology, it's very Fristonian, you know, very active inference-y kind of jargon.

So, I wanted to make sure that they got that from the physics point of view.

Thanks.

Yeah.

Yeah, totally.
Great point.
And I'll ask a question now from Bert.
So, Bert says, very impressive.
Solving generative models with more generative models sounds very promising.
What about replacing ANTs with convolutions?
Joking about, like, a meta-learning algorithm.
Like having a neural network that learns how to optimize other neural networks.
Yeah.
I mean, this concept, I think, is introduced at some point in machine learning lore.
But we wanted to apply for nature-based method that...
It's like the mother nature, which is the...
You know, nature is the most efficient optimization evolution system.
So, looking at the results that are there in those nature, applying nature-based methods,
they were superior to any other method.
And the results we got also, which just pointed out to that...
Pointed that out, that we saw good performance coming out from these results,
from our results and previous results from other methods as well.
So, I'll pop in here a little bit, too.
In the case of neural architecture research, there's kind of a couple classes of approaches.
One is constructive.
So, kind of like where you build larger and larger networks and try and keep your network size minimal
to try and find your optimal solution.
Other types of neural architecture search approaches use, like, a superstructure.
And this is kind of how the earlier iterations of this were, where you have a bound of your search space
and you try and find the optimal network within that bound.
So, one is, like, trying to build things from the ground up,
and the other one's trying to, like, trim down a big network to a small network.
As to your question about convolutions, there's a type of...
There's been a fair bit of research lately in what's called graph-based neural networks,
which can use convolutions over, like, a discrete graph search space
and can potentially produce other graphs.
And I believe there's been some neural architecture search work using this.
But one of the main, and I think cool things about the approach here,
which is different from those, is that even if you have a graph-based neural network
and you have your search space defined as some kind of matrix
where things are off and on depending on which nodes are connected to each other,
you have a fixed search space, which may not be big enough,

or it might not be the correct search space for this neural architecture search problem,
where this method here is all continuous, right?
So, within this continuous search space, you really have...
It gives the algorithm a lot of freedom, maybe too much freedom,
but a really open-ended way of generating a wide variety of neural architectures,
which, if you pre-constrain your algorithm to work within a fixed, discrete superstructure,
you may not even find them because they're not even a possibility.
So, that's one of the reasons we didn't go that route.
But there are graph-based neural architecture search algorithms out there
where basically you take the architecture as a graph,
you train a neural network to spit out another graph that it might think is better.
And those use convolution sometimes.
That helps.
That's awesome.
So, convolution sounds like yes.
And one thought on that is,
yes, the ants solve all these incredible patterns
and kind of do amazing things amidst informational and physical limitations,
like we all do.
Having the ants be able to
just make trade-offs within a task space
and then have a dial as modelers
to make that task space kind of like touching the pheromone distribution
or metacognitive ants
or something emulating essentially that
and, for example, the active inference,
forward-looking and thinking through other minds,
that there could be a kind of cognitive colony.
So, then that enables in silico,
total thought experiment colonies,
and through data-driven processes
also kind of keep continuity with that model,
perhaps literally continuity with the model,
and then connect it to empirical,
which is something that is very hard for agent-based modeling,
which, as you kind of pointed to,
often sets certain fixed axes,
performs like a sweep,

looks at one mechanism,
doesn't look at all these possible mechanisms
of like learning and intra- and intergenerational
and all these time effects.
So, how do you see this being used
in different research or application domains?
Well, I mean, we were...
The main use case that we're thinking of
was New Revolution New Architecture Search.
That's for...
To apply for other domains
other than New Architect Search,
we didn't figure out that yet.
I think Alex can expand on that.
I can hop in a little bit, too.
I mean, so basically,
this type of algorithm,
if you need to generate graphs
and you don't necessarily have a fixed structure
for that graph...
And when I say graph,
I mean, like, a computer science graph
where you have nodes and edges.
So, pretty much anything involving graph construction,
I think these types of methods...
No networks kind of under the hood are...
You know, can be represented as graphs
and usually are.
So, I think we're using it for neural networks
because it's really popular.
But, you know, there's other algorithms out there,
like, you know, the traditional traveling salesman.
There's like routing problems,
all that, any type of stuff
where you might need to generate a graph
in a smart way to do that.
To your other point, though,
I think what's really cool here,

and I don't want to steal outdoor elements thunder,
but, you know, his last point on future direction is,
while a particular version
of this ant colony optimization search
is running to find an optimal neural network,
a colony has fixed parameters
that it operates within.
But if you think of the colony as an organism,
as opposed to the ants being an organism,
you can evolve colonies
that optimize how the ants themselves act.
So, you can have evolving colonies
that, in a smaller sense,
also evolve or optimize
what they're doing
within their prescriptive parameters
for the agents they're generating.
So, you can have like a meta meta.
It's awesome.
I mean, the evolutionary account
of a why question for ant behavior today,
one part of that answer is like,
because colonies that couldn't
under that regularity or constraint survive,
we've had a long, long time
to wipe those off the table.
And so, every biological system
has to have that kind of multi-scale ordering.
In 2021, we made the Active Emperor Ants paper,
which was modified
from an epistemic foraging visual attention task
about scanning around
and then learning a saccade policy
that had to do with epistemic foraging,
but not leaving a trace.
And then the main modification
to bring that active inference,
epistemic visual foraging model

into the active inference
ant setting
was to add a pheromone rule,
just like you described,
even though, of course, again,
that's not the only pheromone rule,
but that's just the most simple pheromone rule
that we can generalize from,
as you definitely have.
And there are just many emerging ways
of modeling those multi-scale active inference models.
So, composing across layers,
which we might associate more
with the kind of laterality
of things that happen through interactions.
And then also, as Mike Levin shows,
with kind of the time diamond systems
that have a memory retention component
of some shape, cognitive shape,
and then a protention awareness
or agency or other attributes
you can use to describe that.
And that that's a statistically amenable way
to describe things.
And then there's a variety of implementations
on a given statistical problem
or like federated compute architecture.
It might be the case
that you're not running
like the pure matrix multiplications
that are shown in like the early MATLAB code
of active inference.
Different components of machine learning systems
might be kind of composed together.
Also ways kind of abiding
by those patterns of communication.
But then there's a level of abstraction
that we can still describe,

but it doesn't mean active inference
is going to be like kind of causing it.
So that's what gives a lot of flexibility.
And it's really cool that through your background,
Alexander, and work,
and these kinds of collaborations
that like the active inference perspective
on multi-agent modeling
with all these other views
can at least come together comparatively.
And then that is going to, I think,
be quite an interesting interchange
to apply this entire tissue type
or colony type,
thinking above and within the models.
Just a lot of degrees of freedom.
Like you said, could be too much.
I mean, and I think there's different ways too.
I think it depends on how you want to take
the ant metaphor.
And, you know, again,
it's kind of interesting
some of the questions or comments
that you're making, Daniel,
and some from the audience about,
you know, how does this, you know,
think about it from a cognitive point of view.
I mean, I do work in cognitive architectures,
of course, again,
kind of from the whole single agent,
you know, or single entity
and, you know, modeling a single brain
in its different regions.
But I think if you take, you know,
a nature inspired optimization approach,
like the ant metaphor
that Abdelrahman, you know,
you know, latched onto.

And that's sort of like the way
in which he formalizes how,
or it takes a principle
of how ants interact with their world,
interact with each other,
and then, you know,
mathematically model
those particular concepts step by step.
And I think if you bend the metaphor
and say, well, okay,
could the ant colony metaphor
apply to multi-human agent systems, right?
Or other entities, does it, does it,
does the ant necessarily,
can it be generalized beyond,
you know, the physical creature
upon which Abdelrahman
based his initial metaphor?
And that's an interesting
philosophical kind of take to it.
And then how do you apply that to,
let's say, building a multi-agent
cognitive system?
And then, of course,
as Travis was discussing with you,
and you were mentioning metacognition,
and, you know,
you could think of ant colonies
of ant colonies,
but you could even replace the word ant
and just say, well,
we have, you know,
clusters of, you know,
intelligent agents,
or however,
whatever degree of modeling we're doing.
Because, again,
I do want to emphasize

that at least the Kants
and Ants agents
that I have worked with
in the context of Abdelrahman,
you know,
they are not each and of themselves.
Even, I would argue,
if nothing else,
a very extremely simplified
generative model,
or a very, very,
very simple control system,
there's no neural network
under each one
because then you'd have
to simulate computationally
each one of these ants
within the frameworks.
I think there's always
that good practical
machine learning
kind of viewpoint of,
well, you know,
there's always a,
how do you simulate that?
And, you know,
Abdelrahman's working with CPUs.
It's not like he has
an army of GPUs
to replace them
with convolutional networks.
Again, if you had the resources,
this would be awesome.
But, you know,
expense and money
is another constraint
on this planet.
But I think there's

interesting views
and interesting directions
one could go
by taking inspiration
from the ant metaphor
and, you know,
the concept of pheromones
and translate them
to other, you know,
other real world signals
and how, for example,
communication patterns
among, you know,
other animal entities
or other human agents.
And I think that that opens up
an interesting perspective.
And if you're constantly
trying to connect it back
to, you know,
free energy minimization
and trying to say,
well, how are we balancing,
you know,
the terms that you can
decompose it into
an epistemic
and an instrumental
and how are these
balancing out
and how are these
physical processes
that we specify,
you know,
balancing those terms.
That's just a very
interesting place to be.
And you mentioned,

you know,
active inference versions
of ants
and that's fascinating
in and of itself.
Last comment I have
is, again,
the degree of modeling
and what you are modeling.
Like if you're modeling
a society
or organization,
you know,
that's one way
you could use,
you know,
the ant colony framework,
if you will,
or optimization
or meta heuristic
optimization frameworks
to, you know,
then cast,
you know,
you know,
a system,
any type of complex
multi-agent system
as an active inference
kind of,
you know,
engaging process.
Or you could go
really low level
and think about,
you know,
cells in a body
or,

you know,
or units that make up
organs or organelles
and trying to say,
well,
can we use this
to model that level
of granularity
within like a human
or an animal entity,
right?
And I think there's
some fascinating questions
about how does this
metaphor manifest itself
at different timescales
and different degrees
of,
you know,
perspective,
right?
About how you're modeling,
what are you looking at?
What's the picture
that you want to emulate?
And of course,
there's always under the hood
this practical consideration
of,
well,
okay,
the computational expense
that you allocate
and are you able
to actually run
that simulation
long enough?
Because I think,

Abderram,
and you can correct me
if I'm wrong,
you mentioned like
one of the experiments
I think was
for the bigger systems
took a month,
right?
Of course,
this was on CPUs.
You know,
that can get pretty
prohibitive
if you want to go
even bigger than that.
But again,
I think it just depends
on, you know,
what hardware you have
to simulate design.
Yeah.
Yeah.
And also,
I mean,
using high-performance
computing,
it's not always
feasible for smaller.
So if we try
to model
the brain of ANS
like small neural network,
using a GPU
might not be
like feasible solution.
Travis is a high-performance
computing specialist

here,
an expert,
tell you,
but like sending
data to a GPU
and getting it back,
it's very time consuming
and resources consuming.
So it will worsen
the time consumption
rather than solving it
because communicating
between the main memory
and the GPU
would have like
an overhead.
So it has to be
a big enough problem
to actually
utilize the GPU
in such solutions.
Yeah.
So we've been,
oh,
sorry.
I forgot to cut you off.
Go ahead.
Go ahead.
Yeah.
So when you have
like,
you know,
the super large language
models
or large models
for computer vision,
they do a lot
of just massive

operations on tensors,
which are basically
multidimensional matrices,
right?
And when you have
really big,
wide tensors,
you can parallelize
the operation
really nicely
across the GPU.
A lot of this work
is based on
doing like
the time series
forecasting,
time series
classification
on sensor data,
stuff from like
power systems.

you know,
the input
to a large language
model might be
a thousand or more,
you know,
word embedding,
length of a word
embedding,
which is actually
not huge,
but if you go up
to computer vision model,
the input image
may be a thousand
by a thousand pixels

and that gives you
like actually
a million inputs,
right?
When you're working
with sensor systems
off of aircraft,
power plants,
that type of thing,
you may have
50 to 100 inputs.
And when you're working
with this type
of time series data,
you don't need
a massive,
super wide
neural network.
And then if you
add in recurrency
where you have to do
backprop over time
and other things like this,
you actually can't
really parallelize it
nicely on a GPU.
So for us,
the CPU is actually
more efficient.
We tried,
Abdulrahman wrote a bunch
of code a long time ago
to put this stuff
on GPUs
and we found
it was quite a bit slower.
So depending on
what you're doing

with a neural network,
a GPU actually
isn't the right answer.
But the other cool
thing about this,
which I think does have
maybe even potential
for there,
is that,
you know,
one of the big
not talked about
problems in machine
learning is that
backpropagation
is the fastest
thing we know,
but it's inherently
not scalable.
Like,
you can get a bigger,
better GPU
to do your bits
of your network
in parallel
to speed things up,
but that only gets better
if you have a bigger network.
If you want to speed up
the training process,
you can't just add
another CPU
or another GPU
and make backprop
go faster.
You can make
the forward and backward
pass through your neural

network faster,
but you still have to do
every epoch
of backprop iteratively.
A method like this
where we generate,
we're not,
we're not,
one,
it's backprop free,
so it's not using backprop.
We can use a nature
inspire or other method
to use hundreds
of computers
and you can throw
twice as many computers
at it and get a result
back twice as fast,
whereas backprop,
you can't do that.
So if you think about
actually being able
to train a neural
network faster,
backprop actually,
you know,
it's got a pretty,
pretty low speed limit
for what we need to do.
And it's kind of
a big problem
of the machine learning
community that people
don't like to talk about
because they're like,
oh,
I'll just buy the next

big NVIDIA GPU
and that'll do things faster.
So that's super interesting.
Does this maybe even
bring up a kind of
relationship where
things like a graphics
visualization,
of course,
a GPU does well.
And that's like the screen
changing through time
with a classical process
that can be massively
unfolded.
and then the cognitive
models ultimately
of the nest mates,
which again can be nested,
but the thing that's
more quantum,
more cognitive model
like you can do
in parallel
because the minds
are not influencing
each other
except through stigmargy.
So then that is CPU bound,
the size of the colony.
And then you could use
different like graphics,
techniques,
like there are colonies,
organisms.
So one ant being,
or it's a philosophical question,
what is the scale

at which A exists,
but all throughout California
with the Argentine ant,
for example.
And so how do we deal
with those kinds of like
meshwork,
cognitive systems,
all the way on through
50 in an acorn?
There's just all these
different trade-offs
that are being made
and like in the
featureless deserts,
there's different
wayfinding,
pathfinding,
sensor integration,
polarization of light,
like different
cognitive strategies
because they might be
going out
long distance
and dragging something home,
not leaving any pheromone
because it's not
any more likely
to have food there.
So in that case,
the stigma G
is basically minimal
to essentially none.
And then in other situations,
you could have
something that's
very adherent

to distributions
to the point of being
like fit
to very
like
kind of
normative path.
But that's happening
at a level
that allows
the
different compute
architectures,
different information
architectures,
and ultimately
different biological
embodiments
to really
engage
fruitfully.
Again,
looking at the
variability,
the diversity
of biological
algorithms
for collective
behavior,
which have been
studied by
Professor Gordon
and others
in so many
different angles,
yet sometimes
it can feel like
multi-agent models

always start
kind of like
at square one,
demonstrate
some proof
of concept
phenomena,
and that is
utilized as part
of like a
bigger perspective,
but it's not
like that
model was
ever like
claimed to
have been
tuned to
maximum
performance.
It's like,
well,
we got
decision-making
behavior.
You could
transfer this
to group
decision-making
with honeybee
decision-making
or something
like that.
But there's
still like
a big gap
there.
But I think

what you're
describing with
the can'ts,
which is funny
because it
could be
cannot,
but also
the can't
is the
dialect,
which is
spoken.
It was
very funny
when we
came up
with it.
Great choice.
And it's
just like,
yeah,
because there's
multiple
perspectives to
swap from
on the
classical screen
because the
meaning of the
word is
something that's
happening in that
fourth dimension.
Cognitively,
the meaning of
the word isn't
to be found

just on the
blanket,
just on the
interface itself.
That's just the
communication.
And that's
like a bounded
system.
Then if you
model a cognitive
system that
doesn't have
that kind of
a constraint,
so represented
by a map
that has
some kind
of blanket
index,
some kind
of blanketing,
if you don't
embody that
constraint in
the statistical
model,
the map,
you're ignoring
one of the
fundamental
constraints
of modeling
the way that
things happen
in an embodied
fashion.

Maybe there's
some abstract
space for a
certain problem
that's just
like a total
slam dunk.
However,
for full
generality,
at least to
the space
that we
know of
biological
life forms
and their
engagements
and like
ecological
engagements,
not just
like within
one behavior.
That space
is so vast
and there's
so much
to learn
across
different
systems
and then
to,
again,
to abduce
away into
different

information
architectures.
Active
inference
being some
subset or
type of
those.
So it's
awesome work.
Do you have
any last
comments?
Well,
being giving
ability for
brands to
be self-aware
and aware
about this
environment,
it's actually
something that
we implemented
in our last
work with
the BP
free camps.
They are
indirectly aware
about their
environment
and they
are adapting
to the
changes in
their
environment

by inductive
meaning that
they are
evolving using
a genetic
based
algorithm
to just
change their
behavior,
like how
they sense
the world,
the world,
the permanence
when they
take the
steps and
some other
things or
some other
characteristics
they have.
So they
are adapting
but kind
of like
not in
an intelligent
way but
through evolution
if you want
to say.
Actually,
we consider
putting a
brain in
each one of

these agents
but then
again,
as Travis
and Alex
mentioned,
we found
that it
won't be
practical.
Actually,
it would
hinder our
asynchronous
design because
we couldn't
like
penalize
that.
It would
take time
to train
each one
of these
brains as
we evolve
and your
networks.
Awesome.
Alexander or
Travis,
any last
thoughts?
Okay.
Jeez.
I'm just
really happy.
I think

this work
is really
interesting
and it
opens up
a lot
of pretty
cool avenues.
Again,
if we can
get to
the point
where we're
evolving
colonies
that are
producing
ants
and can
see where
that can
go.
So one
of the
big issues
in neural
architecture
research
is the
whole
question
of what
is an
optimal
neural
network.
And what
an optimal

neural
network
is
could be
different
for,
well,
will be
different
for different
tasks.
But not
only that,
even if
it's the
same data
set,
how you're
using that
neural network
could lead
to a less
optimal,
less or more
optimal neural
network,
right?
Depending on
what you're
doing with it,
maybe you
need one
that's more
energy efficient.
Maybe you
don't care
about energy
efficiency or

performance and
you'll take a
slower neural
network,
but you need
more accuracy.
So being able
to have
algorithms which
can automate
this whole
process for us
and tune
them to what
we actually want
to use the
neural network for
is really
important.
And I
think,
one,
just having
ant colony
optimization,
be able to
optimize the
network for a
problem is
great.
But two,
if we can
make it such
that the
algorithm itself
is self-optimizing,
it really can
streamline this

whole process
where right
now if you're
doing machine
learning,
it can be
kind of
miserable.
You make
a neural
network
architecture,
you try it
out, see
how well it
does.
Oh, nope,
that didn't
do so well.
Let me tweak
a couple
knobs.
Automating that
process.
My whole
life as a
computer scientist
is about being
lazy but being
smart about it.
So whatever I
can optimize so
that I don't
have to do it
over and over
again seems
like a good
use of my

time.
So I'll be
smart about
having to do
as little as
possible in the
future.
And I guess
I don't have
too much to
add to that.
I think a
lot of the
good discussion
has happened
already and
talked about
various
implications and
ways of
viewing and
colony optimization
from other
perspectives including
an active
inference point
of view.
So I guess
really more
from a
closing thought
on my end
is that it
will be
interesting or
it is an
interesting
direction to

think about
like I said
earlier or
suggested about
the adaptation
of the
metaphor to
other systems
and what are
you trying to
model and
what's your
goals from a
scientific and
philosophical point
of view?
What are the
questions you
seek to answer?
And I think it
might be very
interesting,
again, given
other developments
and computing
technology and
ways in which
you implement
the parallelization
that I think
is that's what
attracted me the
most to a lot
of these
meta-heuristic
algorithms,
even things
like particle

swarm.
And when I
worked with
Travis many
years ago on
the exam
algorithms,
you saw our
names on that
and working
on that type
of stuff,
the part that
always caught
my attention
is, again,
that ability
to say I
can put these
entities on
different
computing
processing resources
or devices,
and then they
will interact
and exchange
their results
in some way
to try to
optimize
some often
complex
multi-objective
cost function.
And so I
think the part
that we'll see

or that would
encourage the
wider spread
adoption of even
like meta-heuristic
algorithms in
general, not to
say that they
aren't used a lot
in, for example,
the engineering
domains, is,
again, the
development of
parallel computing
processing systems
and I think
exploiting things
like asynchronous
computing.
That was, again,
another angle
that caught my
attention from
Travis.
He's done a lot
of work on
genetic optimization
and neuroevolution
from an
asynchronous point
of view and
how can we
allocate whatever
resources are
available and
distributing them
across global

networks.
I think that
might be the
best shot to
scaling up,
let's say,
with what we
got right now.
There might be,
again, you've
mentioned, Daniel,
quantum technology,
quantum
computing is
another
interesting place
that sort of
changes the
game.
But barring
technologies that
we don't
necessarily have
exactly at
their best at
this moment,
how can we
take advantage
of citizen
science or
distributed
computing or
peer-to-peer
type of
communication and
building massive
active inference
systems that

embody the
multi-agent
metaphor of
ant-colony
optimization or
other nature
inspired
frameworks?
Can this
system evolve
over very
long spans
of time?
Just like
evolution really
worked.
Another piece,
my final
comment,
is that why
I'm interested
sometimes in
evolution,
is that to
me, it is
the inductive
bias that
provided us
with structures
that allow,
for example,
a human agent
to operate
already.
Babies can
already recognize
faces.
We have

certain
instinctual
reactions and
certain mechanisms
that evolution
has endowed us
with.
A fascinating
question is,
what is the
interplay of
the idea of
simulating an
artificial form of
evolution, maybe
building DNA
structures or
very, very
simplified
computational
structures, which
would answer
Abdel Rahman's
concern about,
well, maybe we
don't want the
agents to be too
smart enough
themselves because
I can't really
simulate that unless
you give me a
decade to run
the simulator.
But you could
maybe come up
with a more
fundamental primitive

and then use
that as a
starting point for
your neural
network.
Let's say,
Daniel, you
want to do
some task in
image segmentation
and you're
like, okay, but
what can your
evolutionary framework
give me?
Well, I'll say,
here, here's a
template to start
from.
This is a
kernel on which
you build your
framework and
it's like a
DNA structure.
And this has
evolved across
many, many
years of
distributed peer
to peer
computing.
And you can
imagine building
this mammoth
evolving,
continual learning
style, you

know, evolutionary
algorithm, whether
it is based on
genetic algorithms
or ant colony.
And you can
imagine that might
be an interesting
way to think
about it.
And by the
way, I am
spitballing and
generating an
idea of like how
I could envision
a scalable form
without inventing
a new computing
system that I
don't know will
or will not
exist.
Because quantum
has a lot of
problems still to
solve too, like
superconducting
or super
temperatures or
trapping photons,
as I have
learned.
So that might be
an interesting
direction.
I think the
scaling of this,

especially from
the practical
end, is going
to be the most
important.
We're going to
need to pull
together all the
tools that we
have, as I
mentioned before.
So I'll stop
there too, because
I'll end up
rambling more.
So hopefully
that made
sense.
Well, this was
very epic and
inspiring.
So good luck
with the work.
You're all
welcome to
suggest another
piece that we
might focus on
or continue the
discussion however
you see fit,
because it's
super interesting
direction.
So thank you.
Until next time.
Bye.
Thank you so

much,

Travis.

Bye.

Bye.

Bye.

Bye.

Bye.